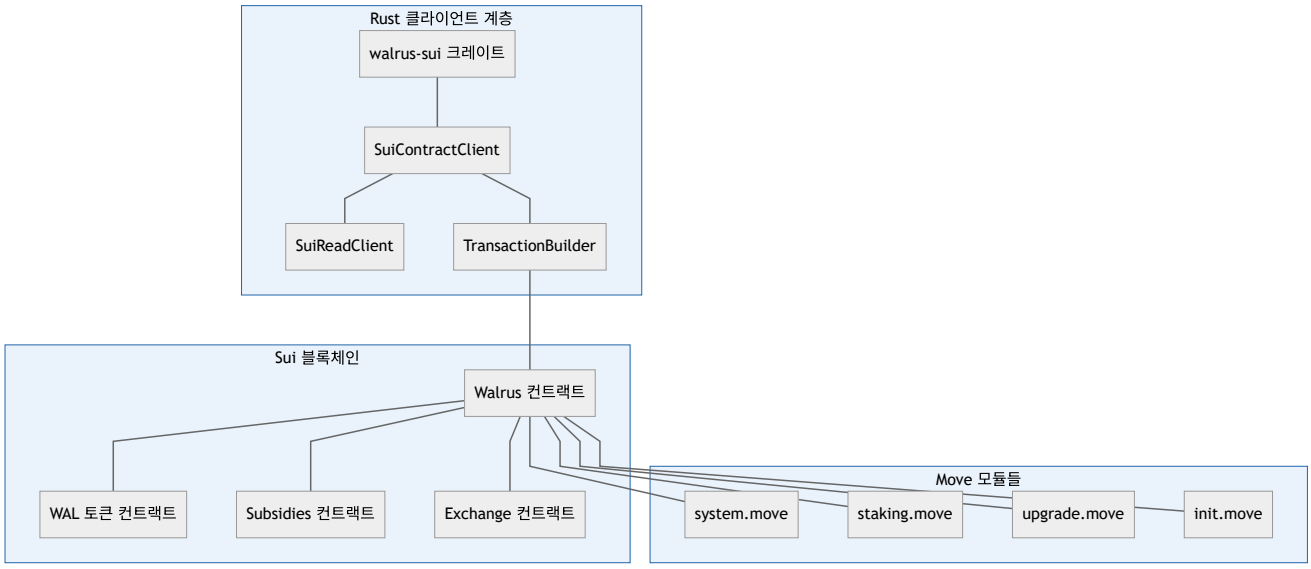
Sui 블록체인 통합 메커니즘 분석

개요

Walrus는 Sui 블록체인을 기반으로 한 탈중앙화 블롭 스토리지 시스템으로, 스마트 컨트랙트를 통해 거버넌스, 스테이킹, 결제 시스템을 관리합니다. 본 문서는 Walrus가 Sui 블록체인과 통합되는 메커니즘을 상세히 분석합니다.

아키텍처 개요



walrus-sui 크레이트 분석

핵심 컴포넌트

1. SuiContractClient

```
pub struct SuiContractClient {
inner: Mutex<SuiContractClientInner>,
pub read_client: Arc<SuiReadClient>,
wallet_address: SuiAddress,
gas_budget: Option<u64>,
}
```

주요 기능:

- 트랜잭션 실행의 상호 배제 보장
- 동일한 소유 객체의 동시 사용 방지
- 가스 예산 관리 및 자동 추정

2. SuiReadClient

```
pub struct SuiReadClient {
retriable_sui_client: RetriableSuiClient,
contract_config: ContractConfig,
}
```

주요 기능:

- 블록체인 상태 읽기 전용 접근
- 컨트랙트 설정 관리
- 재시도 로직이 포함된 RPC 클라이언트

3. WalrusPtbBuilder (Programmable Transaction Builder)

```
pub struct WalrusPtbBuilder {
pt_builder: ProgrammableTransactionBuilder,
read_client: Arc<SuiReadClient>,
tx_wal_balance: u64,
tx_sui_cost: u64,
used_wal_coins: BTreeSet<ObjectID>,
wal_coin_arg: Option<Argument>,
}
```

주요 기능:

- 복합 트랜잭션 구성
- WAL 코인 자동 병합 및 관리
- 가스 비용 추적

Move 컨트랙트 구조 분석

1. 시스템 컨트랙트 (system.move)

핵심 구조체

```
public struct System has key {
id: UID,
version: u64,
package_id: ID,
new_package_id: Option<ID>,
}
```

주요 기능

블롭 관리:

```
// 스토리지 공간 예약
public fun reserve_space(
self: &mut System,
storage_amount: u64,
epochs_ahead: u32,
payment: &mut Coin<WAL>,
ctx: &mut TxContext,
): Storage

// 블롭 등록
public fun register_blob(
self: &mut System,
storage: Storage,
```

```
blob_id: u256,
root_hash: u256,
size: u64,
encoding_type: u8,
deletable: bool,
write_payment: &mut Coin<WAL>,
ctx: &mut TxContext,
): Blob

// 블롭 인증
public fun certify_blob(
self: &System,
blob: &mut Blob,
signature: vector<u8>,
signers_bitmap: vector<u8>,
message: vector<u8>,
)
```

에포크 관리:

```
public(package) fun advance_epoch(
self: &mut System,
new_committee: BlsCommittee,
```

```
new_epoch_params: &EpochParams,

): VecMap<ID, Balance<WAL>>
```

2. 스테이킹 컨트랙트 (staking.move)

핵심 구조체

```
public struct Staking has key {

id: UID,

version: u64,

package_id: ID,

new_package_id: Option<ID>,

}
```

주요 기능

노드 등록:

```
public fun register_candidate(

staking: &mut Staking,

name: String,

network_address: String,

metadata: NodeMetadata,

public_key: vector<u8>,

network_public_key: vector<u8>,

proof_of_possession: vector<u8>,
```

```
commission_rate: u16,
storage_price: u64,
write_price: u64,
node_capacity: u64,
ctx: &mut TxContext,
): StorageNodeCap
```

스테이킹 관리:

```
// WAL 스테이킹
public fun stake_with_pool(
staking: &mut Staking,
to_stake: Coin<WAL>,
node_id: ID,
ctx: &mut TxContext,
): StakedWal

// 스테이킹 철회 요청
public fun request_withdraw_stake(
staking: &mut Staking,
staked_wal: &mut StakedWal,
ctx: &mut TxContext,
)
```

```
// 스테이킹 철회
public fun withdraw_stake(
staking: &mut Staking,
staked_wal: StakedWal,
ctx: &mut TxContext,
): Coin<WAL>
```

에포크 변경:

```
// 투표 종료
public fun voting_end(staking: &mut Staking, clock: &Clock)

// 에포크 변경 시작
public fun initiate_epoch_change(
staking: &mut Staking,
system: &mut System,
clock: &Clock
)

// 에포크 동기화 완료
public fun epoch_sync_done(
staking: &mut Staking,
cap: &mut StorageNodeCap,
```

```
epoch: u32,
clock: &Clock,
)
```

3. 업그레이드 컨트랙트 (upgrade.move)

핵심 구조체

```
public struct UpgradeManager has key {
id: UID,
cap: UpgradeCap,
upgrade_proposals: Table<PackageDigest, UpgradeProposal>,
}

public struct EmergencyUpgradeCap has key, store {
id: UID,
upgrade_manager_id: ID,
}
```

업그레이드 프로세스

```
// 업그레이드 투표
public fun vote_for_upgrade(
self: &mut UpgradeManager,
```

```
staking: &Staking,

auth: Authenticated,

node_id: ID,

digest: vector<u8>,

)


// 쿼럼 기반 업그레이드 승인

public fun authorize_upgrade(

self: &mut UpgradeManager,

staking: &Staking,

digest: vector<u8>,

): UpgradeTicket


// 긴급 업그레이드 승인

public fun authorize_emergency_upgrade(

upgrade_manager: &mut UpgradeManager,

emergency_upgrade_cap: &EmergencyUpgradeCap,

digest: vector<u8>,

): UpgradeTicket
```

에포크 관리 메커니즘

에포크 생명주기

에포크 매개변수 계산

투표 집계 로직:

```
// 가중 중앙값 계산 (스테이킹 가중치 기반)
pub fn compute_weighted_median(votes: &[(u64, u64)]) -> u64 {
// votes: (value, weight) 쌍들
let total_weight: u64 = votes.iter().map(|(_, w)| w).sum();
let target_weight = total_weight / 2;
let mut cumulative_weight = 0;
for (value, weight) in votes.iter() {
cumulative_weight += weight;
if cumulative_weight >= target_weight {
return *value;
}
}
unreachable!()
}
```

스테이킹 메커니즘

스테이킹 풀 구조

```
pub struct StakingPool {
pub node_id: ObjectID,
pub staked_amount: u64,
pub rewards: Balance<WAL>,
```

```
pub  commission_rate: u16,

pub  delegators: VecMap<SuiAddress, StakedWal>,

}
```

보상 계산

```
public fun calculate_rewards(

staking: &Staking,

node_id: ID,

staked_principal: u64,

activation_epoch: u32,

withdraw_epoch: u32,

): u64 {

// 에포크별 보상률 적용

let mut total_rewards = 0;

for epoch in activation_epoch..withdraw_epoch {

let epoch_reward_rate = get_epoch_reward_rate(epoch);

let commission_rate = get_commission_rate(node_id, epoch);

let net_rate = epoch_reward_rate * (1 - commission_rate);

total_rewards += staked_principal * net_rate;

}

total_rewards
```

```
}
```

거버넌스 메커니즘

권한 관리

```
// 인증 시스템
public struct Authenticated has drop {
sender: address,
}

public struct Authorized has copy, drop, store {
inner: AuthorizedInner,
}

enum AuthorizedInner has copy, drop, store {
Address(address),
Object(ID),
}
```

투표 시스템

업그레이드 제안:

```
public struct UpgradeProposal has drop, store {
```

```
epoch: u32,
digest: PackageDigest,
version: u64,
voting_weight: u16,
voters: VecSet<ID>,
}
```

쿼럼 계산:

```
pub fn is_quorum(&self, weight: u16) -> bool {
// 2f + 1 (비잔틴 장애 허용)
weight > 2 * self.max_faulty_nodes()
}
```

트랜잭션 생성 및 실행 과정

1. 트랜잭션 구성

```
impl WalrusPtbBuilder {
pub async fn reserve_and_register_blob(
&mut self,
blob_metadata: BlobObjectMetadata,
epochs_ahead: EpochCount,
persistence: BlobPersistence,
```

```
) -> SuiClientResult<Argument> {
// 1. 스토리지 공간 예약
let storage_arg = self.reserve_space(
blob_metadata.encoded_size,
epochs_ahead
).await?;
// 2. 블롭 등록
let blob_arg = self.register_blob(
storage_arg.into(),
blob_metadata,
persistence,
).await?;
Ok(blob_arg)
}
}
```

2. 트랜잭션 서명 및 전송

```
impl SuiContractClient {
async fn execute_transaction(
&self,
ptb: ProgrammableTransaction,
) -> SuiClientResult<SuiTransactionBlockResponse> {
```

```
let mut inner = self.inner.lock().await;

// 1. 트랜잭션 데이터 생성

let tx_data = inner.build_transaction_data(ptb).await?;

// 2. 서명

let signed_tx = inner.wallet.sign_transaction(&tx_data)?;

// 3. 전송 및 실행

let response = inner.read_client

.retriable_sui_client()

.execute_transaction_block(

signed_tx,

SuiTransactionBlockResponseOptions::full_content(),

Some(ExecuteTransactionRequestType::WaitForLocalExecution),

)

.await?;

// 4. 실행 결과 검증

self.verify_transaction_success(&response)?;

Ok(response)

}

}
```

이벤트 모니터링 시스템

이벤트 타입 정의

```rust
#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserialize)]
pub enum ContractEvent {
BlobEvent(BlobEvent),
EpochChangeEvent(EpochChangeEvent),
PackageEvent(PackageEvent),
DenyListEvent(DenyListEvent),
ProtocolEvent(ProtocolEvent),
}

#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserialize)]
pub enum BlobEvent {
Registered(BlobRegistered),
Certified(BlobCertified),
Deleted(BlobDeleted),
InvalidBlobID(InvalidBlobId),
DenyListBlobDeleted(DenyListBlobDeleted),
}
```

이벤트 스트리밍

```rust
impl SuiReadClient {
pub async fn subscribe_to_events(
&self,
```

```
event_filter: EventFilter,
) -> Result<impl Stream<Item = SuiEvent>, SuiClientError> {
self.retriable_sui_client
.subscribe_event(event_filter)
.await
.map_err(SuiClientError::from)
}
pub  async  fn  query_events(
&self,
query: EventFilter,
cursor: Option<EventID>,
limit: Option<usize>,
descending_order: bool,
) -> SuiClientResult<EventPage> {
self.retriable_sui_client
.query_events(query, cursor, limit, descending_order)
.await
.map_err(SuiClientError::from)
}
}
```

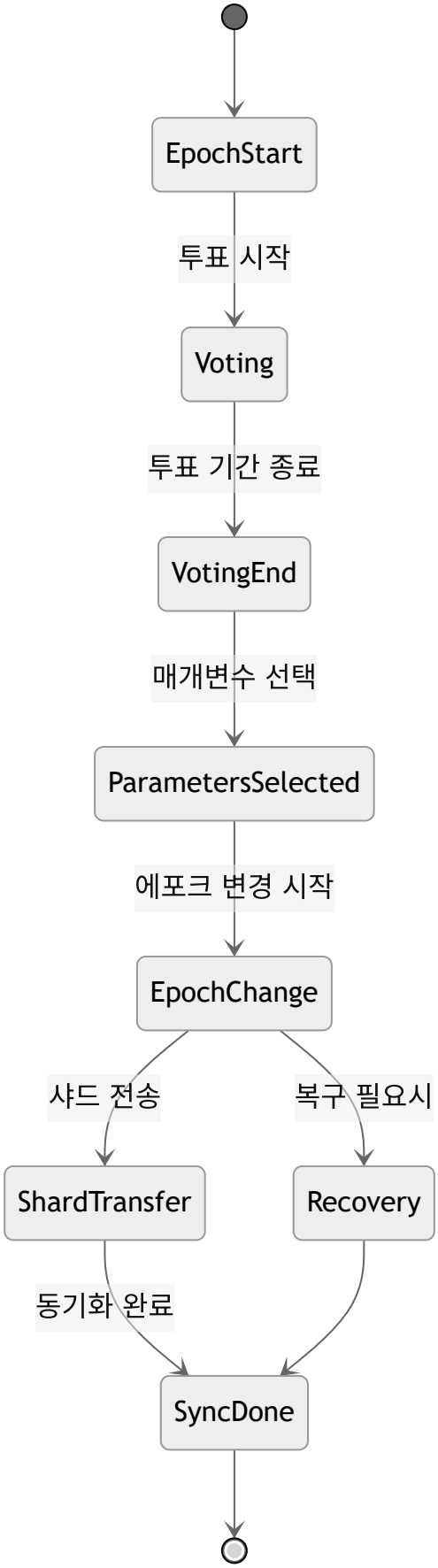
상태 동기화 메커니즘

위원회 상태 동기화

```
impl Committee {
pub fn compare_functional_equivalence(
&self,
other: &Committee
) -> anyhow::Result<()> {
// 에포크 비교
if self.epoch != other.epoch {
return Err(anyhow!("Epoch mismatch"));
}
// 샤드 수 비교
if self.n_shards != other.n_shards {
return Err(anyhow!("Shard count mismatch"));
}
// 노드별 샤드 할당 비교
for (node_id, left_node) in &self.members {
if let Some(right_node) = other.find(node_id) {
if left_node.shard_ids != right_node.shard_ids {
return Err(anyhow!("Shard assignment mismatch"));
}
} else {
return Err(anyhow!("Node not found in other committee"));
```

```
}

}

Ok(())

}

}
```

에포크 동기화 프로세스



WAL 토큰 경제학

WAL 토큰 구조

```
module wal::wal;

const TOTAL_WAL_SUPPLY_TO_MINT: u64 = 5_000_000_000; // 5B WAL
const DECIMALS: u8 = 9;

public struct ProtectedTreasury has key {
id: UID,
// TreasuryCap은 동적 필드로 저장
}
```

가격 계산 메커니즘

```
pub fn storage_price_for_encoded_length(
encoded_size: u64,
epochs: EpochCount,
storage_price_per_unit: u64,
) -> u64 {
let size_units = (encoded_size + STORAGE_UNIT_SIZE - 1) / STORAGE_UNIT_SIZE;
size_units * epochs as u64 * storage_price_per_unit
}

pub fn write_price_for_encoded_length(
encoded_size: u64,
```

```
write_price_per_unit: u64,

) -> u64 {

let  size_units = (encoded_size + WRITE_UNIT_SIZE - 1) / WRITE_UNIT_SIZE;

size_units * write_price_per_unit

}
```

보조금 시스템 (Subsidies)

Walrus Subsidies 구조

```
public struct WalrusSubsidies has key {

id: UID,

version: u64,

package_id: ID,

}


public struct AdminCap has key, store {

id: UID,

subsidies_id: ID,

}
```

보조금 처리 로직

```
public fun process_subsidies(
self: &mut WalrusSubsidies,
staking: &mut Staking,
system: &mut System,
clock: &Clock,
) {
// 1. 시스템 보조금 계산
let system_subsidy = calculate_system_subsidy(self, system);
// 2. 노드별 보조금 계산
let node_subsidies = calculate_node_subsidies(self, staking);
// 3. 보조금 분배
system.add_per_epoch_subsidies(system_subsidy);
staking.add_commission_to_pools(node_ids, node_subsidies);
}
```

오류 처리 및 복구

트랜잭션 오류 처리

```
#[derive(Debug, thiserror::Error)]
pub  enum SuiClientError {
#[error("credits are not enabled for this client")]
CreditsNotEnabled,
```

```
#[error("walrus subsidies are not configured for this client")]
WalrusSubsidiesNotConfigured,
#[error("transaction execution failed: {0}")]
TransactionExecutionError(MoveExecutionError),
#[error("could not find WAL coins with sufficient balance")]
NoCompatibleWalCoins,
#[error("execution cancelled due to shared object congestion on objects {0:?}")]
SharedObjectCongestion(Vec<ObjectID>),
}
```

재시도 메커니즘

```
impl SuiContractClient {
async fn retry_on_wrong_version<F, Fut, T>(&self, f: F) -> SuiClientResult<T>
where
F: Fn() -> Fut,
Fut: Future<Output = SuiClientResult<T>>,
{
const MAX_RETRIES: usize = 3;
for attempt in 0..MAX_RETRIES {
match f().await {
Ok(result) => return Ok(result),
Err(SuiClientError::SuiError(err)) if is_version_error(&err) => {
```

```
if attempt == MAX_RETRIES - 1 {
return Err(SuiClientError::SuiError(err));
}
// 객체 버전 새로고침
self.refresh_object_versions().await?;
}
Err(err) => return Err(err),
}
}
unreachable!()
}
}
```

성능 최적화

배치 처리

```
impl WalrusPtbBuilder {
pub async fn batch_burn_blobs(
&mut self,
blob_objects: Vec<ObjectID>,
) -> SuiClientResult<()> {
// 최대 1000개씩 배치 처리
```

```
for chunk in blob_objects.chunks(MAX_BURNS_PER_PTB) {
for blob_id in chunk {
self.burn_blob((*blob_id).into()).await?;
}
}
Ok(())
}
}
```

연결 풀링

```
pub struct RetriableSuiClient {
clients: Vec<SuiClient>,
current_client: AtomicUsize,
backoff_config: ExponentialBackoffConfig,
}

impl RetriableSuiClient {
async fn execute_with_retry<F, T>(&self, operation: F) -> Result<T, SuiError>
where
F: Fn(&SuiClient) -> BoxFuture<'_, Result<T, SuiError>>,
{
let mut backoff = self.backoff_config.create_backoff();
```

```
loop {
let  client_index = self.current_client.load(Ordering::Relaxed);
let  client = &self.clients[client_index % self.clients.len()];
match  operation(client).await {
Ok(result) => return Ok(result),
Err(err) if  should_retry(&err) => {
if  let Some(delay) = backoff.next_backoff() {
tokio::time::sleep(delay).await;
// 다음 클라이언트로 라운드 로빈
self.current_client.store(
client_index + 1,
Ordering::Relaxed
);
continue;
} else {
return Err(err);
}
}
Err(err) => return Err(err),
}
}
}
```

```
}
```

보안 고려사항

권한 검증

```
public fun check_governance_authorization(
staking: &Staking,
node_id: ID,
auth: Authenticated,
): bool {
let pool = staking.get_pool(node_id);
match pool.governance_authorized {
Authorized::Address(addr) => addr == auth.sender,
Authorized::Object(obj_id) => {
// 객체 기반 권한 검증
verify_object_ownership(obj_id, auth.sender)
}
}
}
```

서명 검증

```
impl ConfirmationCertificate {
```

```
pub fn verify(
&self,
committee: &Committee,
blob_id: BlobId,
) -> Result<(), CertificateError> {
// 1. 서명자 비트맵 검증
let signers = self.decode_signers_bitmap(committee.n_shards())?;
// 2. 쿼럼 검증
let total_weight = signers.iter()
.map(|&signer| committee.get_node_weight(signer))
.sum();
if !committee.is_quorum(total_weight) {
return Err(CertificateError::InsufficientSigners);
}
// 3. BLS 집계 서명 검증
let message = self.create_message(blob_id);
let aggregate_key = committee.aggregate_public_key(&signers);
if !aggregate_key.verify(&message, &self.signature) {
return Err(CertificateError::InvalidSignature);
}
Ok(())
}
```

```
}
```

결론

Walrus의 Sui 블록체인 통합은 다음과 같은 핵심 특징을 가집니다:

1. **모듈화된 아키텍처**: 시스템, 스테이킹, 업그레이드 컨트랙트의 분리
2. **강력한 거버넌스**: 쿼럼 기반 의사결정과 비잔틴 장애 허용
3. **유연한 에포크 관리**: 동적 매개변수 조정과 위원회 재구성
4. **효율적인 트랜잭션 처리**: PTB를 통한 배치 처리와 최적화
5. **포괄적인 이벤트 시스템**: 실시간 상태 모니터링과 동기화
6. **경제적 인센티브**: WAL 토큰 기반 스테이킹과 보상 시스템

이러한 통합 메커니즘을 통해 Walrus는 탈중앙화된 블롭 스토리지 시스템으로서 확장성, 보안성, 그리고 거버넌스를 효과적으로 달성합니다.